

Practical – Week7

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void code_function() {
    printf("Code address    : %p\n", (void *)code_function);
}

int main() {
    int local_var = 30;
    static int static_local = 40;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 50;

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Static local   : %p\n", (void *)&static_local);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&local_var);

    code_function();

    free(heap_var);

    return 0;
}
```

```
pawan_kumar@LAPTOP-SSD0PUQ2:~$ nano memory.c
pawan_kumar@LAPTOP-SSD0PUQ2:~$ gcc memory.c -o memory
pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./memory
Global address : 0x5ac09ec28010
Static address : 0x5ac09ec28014
Static local   : 0x5ac09ec28018
Heap address    : 0x5ac0d852b010
Stack address   : 0x7ffc951fb51c
Code address    : 0x5ac09ec251a9
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |
```

Report: The C program successfully prints the addresses of code, global, static, heap, and stack variables. The experiment demonstrates the Linux process address space and how different memory segments store and manage data during program execution.

2. Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

int main() {
    int local_var = 20;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 30;

    printf("Process ID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address: %p\n", (void *)&local_var);
    printf("Heap address: %p\n", (void *)heap_var);

    printf("Process is running...\n");

    while (1) {
        sleep(10);
    }

    free(heap_var);
    return 0;
}
```

```

code address : 0x0ae09cc201a9
pawan_kumar@LAPTOP-SSD0PUQ2:~$ cd OS_Lab
-bash: cd: OS_Lab: No such file or directory
pawan_kumar@LAPTOP-SSD0PUQ2:~$ mkdir OS_Lab
cd OS_Lab
pawan_kumar@LAPTOP-SSD0PUQ2:~/OS_Lab$ nano memory.c
pawan_kumar@LAPTOP-SSD0PUQ2:~/OS_Lab$ gcc memory.c -o memory
pawan_kumar@LAPTOP-SSD0PUQ2:~/OS_Lab$ ./memory &
[1] 772
Process ID: 772
Global address: 0x63f4f0aef010
Stack address: 0x7ffe4b5b673c
Heap address: 0x63f51201e010
Process is running...
pawan_kumar@LAPTOP-SSD0PUQ2:~/OS_Lab$ |

```

```

Process is running...
pawan_kumar@LAPTOP-SSD0PUQ2:~/OS_Lab$ cat /proc/772/maps
63f4f0aeb000-63f4f0aec000 r--p 00000000 08:30 13008 /home/pawan_kumar/OS_Lab/memory
63f4f0aec000-63f4f0aed000 r-xp 00001000 08:30 13008 /home/pawan_kumar/OS_Lab/memory
63f4f0aed000-63f4f0aee000 r--p 00002000 08:30 13008 /home/pawan_kumar/OS_Lab/memory
63f4f0aee000-63f4f0aef000 r--p 00002000 08:30 13008 /home/pawan_kumar/OS_Lab/memory
63f4f0aef000-63f4f0af0000 rw-p 00003000 08:30 13008 /home/pawan_kumar/OS_Lab/memory
63f51201e000-63f51203f000 rw-p 00000000 00:00 0 [heap]
77ce3f400000-77ce3f428000 r--p 00000000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
77ce3f428000-77ce3f5c0000 r-xp 00028000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
77ce3f5c0000-77ce3f60e000 r--p 001c0000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
77ce3f60e000-77ce3f612000 r--p 0020d000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
77ce3f612000-77ce3f614000 rw-p 00211000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
77ce3f614000-77ce3f621000 rw-p 00000000 00:00 0
77ce3f6a0000-77ce3f6a3000 rw-p 00000000 00:00 0
77ce3f6a9000-77ce3f6ab000 rw-p 00000000 00:00 0
77ce3f6ab000-77ce3f6af000 r--p 00000000 00:00 0
77ce3f6af000-77ce3f6b1000 r--p 00000000 00:00 0
77ce3f6b1000-77ce3f6b3000 r-xp 00000000 00:00 0
77ce3f6b3000-77ce3f6b4000 r--p 00000000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77ce3f6b4000-77ce3f6e3000 r-xp 00001000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77ce3f6e3000-77ce3f6ee000 r--p 00030000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77ce3f6ee000-77ce3f6f0000 r--p 0003b000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77ce3f6f0000-77ce3f6f1000 rw-p 0003d000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77ce3f6f1000-77ce3f6f2000 rw-p 00000000 00:00 0
7ffe4b597000-7ffe4b5b8000 rw-p 00000000 00:00 0 [stack]
pawan_kumar@LAPTOP-SSD0PUQ2:~/OS_Lab$ |

```

Report: The /proc/[PID]/maps file helps us understand how Linux organizes a process's virtual memory. Memory analysis tools such as pmap, ps, and top help inspect memory usage and mappings of a running process.